LaTeX Pipeline Self-Test

A placeholder document for the RetNet report pipeline

RetNet Project

2026-10-13

Purpose	exercises ”Chinese-only Markdown → Chinese PDF” and ”hand-written supplement → English PDF”
Model	504.18M parameters, 20 layers, 1408 wide, 22 heads (placeholder)
Data	552.33M tokens × 3 epochs (placeholder)

This PDF is a self-test artefact, not a deliverable.

Abstract

This document exists only to exercise the build pipeline. Its Chinese edition is generated from a Chinese-only Markdown file, and this English edition is generated in full from a hand-written supplement. Every number quoted here is copied from the 0.5B fact sheet, which was itself read out of the real `trainer_state.json` files; nothing here is invented.

Contents

Abstract **2**

1 Introduction **4**

1.1 Configuration 4

2 Training **4**

2.1 Training Legs 4

2.2 Validation Loss 4

2.3 The Cost of the Hot Restarts 4

A. Appendix A: Reproduction **7**

1. References **7**

1 Introduction

This supplement is the whole English body of the self-test document. It exists to prove that the English edition of a Chinese-only report can be built from a hand-written supplement, with the same structure as the Chinese edition: the same sections in the same order, the same tables, and the same figure.

The 0.5B model has 504,179,896 parameters (504.18M) in 20 layers of width 1408 with 22 heads, and was trained for three legs over the same 552.33M-token packed corpus, one epoch per leg. The pipeline under test must render all of that without inventing anything.

1.1 Configuration

Table 1. Model configuration

Item	Value
Parameters	**504,179,896** (504.18M)
Layers / width / heads	20 / 1408 / 22
Feed-forward and value width	2816 / 2816
Vocabulary	20,000 (tied embeddings)
Retention mode	`chunkwise`, `chunk_size` 128
Learned per-head decay gate	enabled (`use_gla`)
Growth layer norm	enabled, `growth_old_dim` 768
Storage precision	`float32` in `config.json`; trained in bf16

2 Training

The three legs share one corpus and one schedule shape: every leg restarts a full cosine with a peak of 1.1×10^{-4} and 500 warm-up steps, rather than continuing from the previous leg's learning-rate tail.

2.1 Training Legs

Table 2. The three training legs

Leg	Hardware	Steps	Tokens per step	Total tokens
ep1	RTX 3090 24 GB	67,390	8,192	552,058,880
ep2	RTX PRO 6000	16,848	32,768	552,140,800
ep3	RTX PRO 6000	16,848	32,768	552,140,800

Cumulatively the three legs consumed 1,656,340,480 tokens, or 3.285 tokens per parameter.

2.2 Validation Loss

Table 3. Validation loss of the three legs

Leg	First eval	Best eval	Best step
ep1	2.6358	**2.1035**	65,000
ep2	2.1800	**1.9865**	16,848
ep3	2.1174	**1.9459**	16,848

2.3 The Cost of the Hot Restarts

Table 4. Cost of the hot restarts

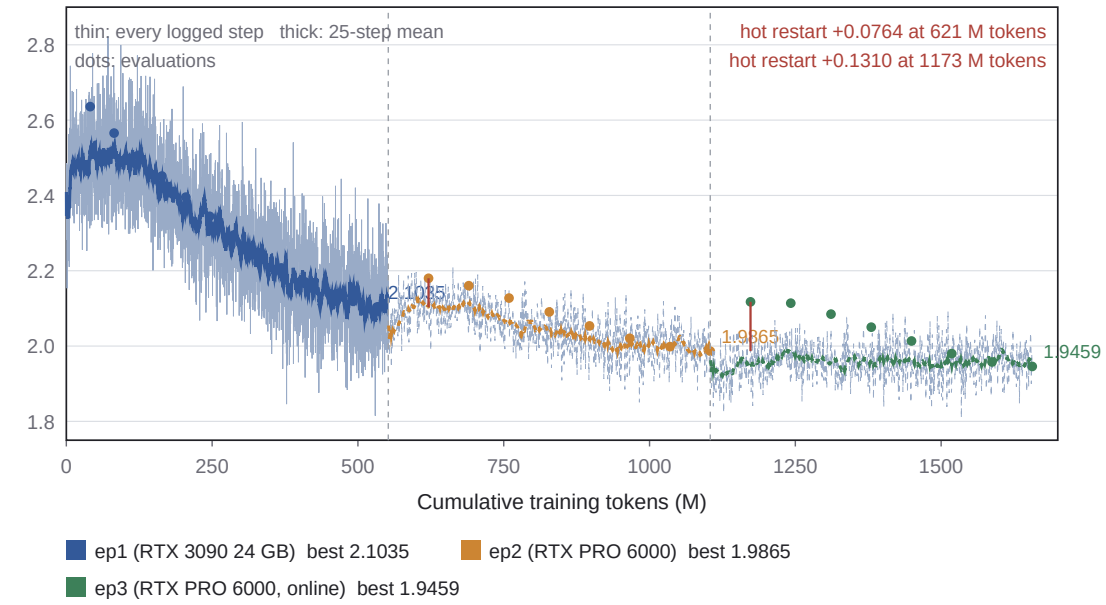


Figure 1. Validation loss of the three training legs. The thin line is every logged step, the thick line its 25-step rolling mean, and the dots are the evaluations; every leg restarts a full cosine. Source: the three `trainer_state.json` files.

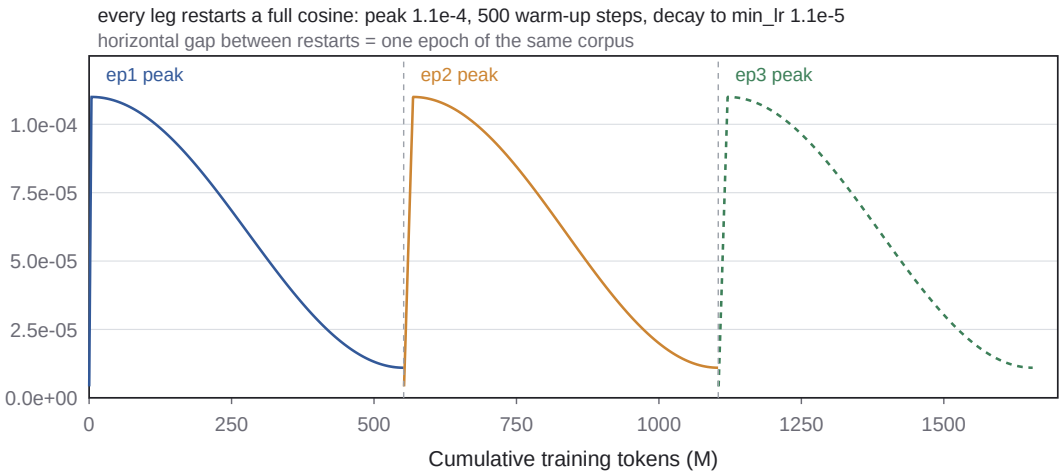


Figure 2. Learning rate of the three legs: every leg restarts a full cosine (peak 1.1×10^{-4}, 500 warm-up steps, decay to `min_lr` 1.1×10^{-5}) instead of continuing from the previous leg's tail. Same source as the loss figure.

Hand-off	**Previous best**	**Next first eval**	**Difference**
ep1 → ep2	2.1035 @65,000	2.1800 @2,106	$+0.0764$
ep2 → ep3	1.9865 @16,848	2.1174 @2,106	$+0.1310$

Both hand-offs made the model *worse* before it recovered: the first evaluation of a new leg sits above the previous leg's best, by 0.0764 and 0.1310 respectively, and both legs do eventually finish below that starting point.

A. Appendix A: Reproduction

```
# 1) the figures, from the real state files
python makefigs.py

# 2) Markdown -> LaTeX, then two XeLaTeX passes per edition
python mktex.py --report selftest
```

1. References

- Retentive Network: A Successor to Transformer for Large Language Models. arXiv:2307.08621.
- Gated Linear Attention Transformers with Hardware-Efficient Training. arXiv:2312.06635.